

CO2_AT2: SCENARIO BASED PROGRAM

7/29/26, 9:36 PM

Student Submissions — Scenario Based Program Writing

SIMATS Engineering • CSE • CSA0804 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

Student 35

Vaibhava Lakshmi K 192511310RD

4 / 4 submitted

Q1. Intelligent Customer Feedback Analysis

CODE

```
def analyse(feedback):
    feedback=feedback.lower().strip()
    positive=["good", "excellent", "happy"]
    negative=["bad", "poor", "worst"]
    p=n=0
    for word in feedback.split():
        if word in positive:
            p+=1
        elif word in negative:
            n+=1
    if p>n:
        print("positive")
    elif n>p:
        print("negative")
    else:
        print("neutral")
num=int(input("enter no of feedback:""\n"))
for i in range(num):
    fb=input("enter feedback:""\n")
    analyse(fb)
```

INPUT

2
good service
worst product

OUTPUT

(no output)

Q2. Comprehensive Password Strength Checker

CODE

```
def check(pwd):
    if not pwd:
        print("strength: weak(empty input)")
        return
    if len(pwd)>64:
        print("strength: weak(too long)")
        return
    u=l=d=s=0
    symbols="!@#%&*()_+~[]{}|;:'.<?/~'"
    for c in pwd:
        if c.isupper():u+=1
        elif c.islower():l+=1
        elif c.isdigit():d+=1
        elif c in symbols:s+=1
    types=(u>0)+(l>0)+(d>0)+(s>0)
    if len(pwd) ≥ 12 and types ==4:
        print("strength: very strong")
    elif len(pwd)≥8 and types ≥3:
        print("strength:strong")
    elif len(pwd)≥6 and types ≥2:
        print("strength:moderate")
    else:
        print("strength:weak")
    if len(pwd)<8:print(" make it atleast 8 characters")
    if not u:print(" add uppercase letters")
    if not l:print(" add lowercase letters")
    if not d:print(" add digits")
    if not s:print(" add special characters")
    num=int(input("enter no of passwords:""\n"))
    for _ in range(num):
        pwd=input("enter password""\n")
        check(pwd)
```

INPUT

```
3
pass
Password1
MYP@ssw0rd2026!
```

OUTPUT

(no output)

Q3. Structured Server Log Error Analyser

CODE

```
def analyse(logs):
    unique_errors=[]
    error_count={}
    for log in logs:
        if log.startswith("ERROR") or "ERROR" in log:
            if log not in unique_errors:
                unique_errors.append(log)
            if log.find("Database")!= -1:
                category="database error"
            elif log.find("network")!= -1:
                category="network error"
            else:
                category="general error"
            error_count[category]=error_count.get(category,0)+1
    return unique_errors,error_count
def print(unique_errors,error_count):
    print("\n--unique error logs--")
    for err in unique_errors:
        print(f"--{err}")
    print("\n--error summary--")
    for category,count in error_count.items():
        print(f"{category}:{count}")
n=int(input("enter no of log lines:"))
logs=[input("enter log line:").strip() for _ in range(n)]
unique_errs,counts=analyse(logs)
print(unique_errs,counts)
```

OUTPUT

(no output)

Q4. Email Domain Frequency Analyser

CODE

```
def analyse_emails(emails):
    counts={}
    for email in emails:
        parts=email.strip().lower().split('@')
        if len(parts)==2 and parts[0] and parts[1]:
            domain=parts[1]
            counts[domain]=counts.get(domain,0)+1
    print("\nDomain frequencies:",counts)
    if counts:
        most_common=max(counts,key=counts.get)
        print(f"Most common:{most_common} [{counts[most_common]}]")
n=int(input("enter no of emails:"))
logs=[input("enter email:").strip() for _ in range(n)]
analyse_emails(logs)
```

INPUT

```
5
alice@company.com
bob@gmail.com
charlie@company.com
invalid-email-format
david@gmail.com
```

OUTPUT

(no output)